
Network Route Optimizer Using Dijkstra's Algorithm

Shortest Path Computation for Weighted Graphs in Networking and Navigation

Project Title	Network Route Optimizer Using Dijkstra's Algorithm
Subject	Design and Analysis of Algorithms (DAA)
Project Type	Algorithm Optimization Project
Difficulty Level	Intermediate
Semester	Semester 5, B.Tech / GSSIPU Curriculum
Author	Mayank Swaraj

Abstract

This paper presents the design and implementation of a network route optimizer built on Dijkstra's shortest path algorithm. Given a weighted graph representing a network of interconnected nodes, the system computes the minimum-cost path from a designated source node to every other reachable node, resolving the core routing problem underlying applications such as GPS navigation, internet packet routing, and transportation network planning. This paper documents the graph-theoretic foundation of the algorithm, walks through a worked example on a four-node weighted graph, presents an array-based C++ implementation, and analyzes the time complexity of both the basic array implementation and the asymptotically faster priority-queue variant.

Keywords: Dijkstra's algorithm, shortest path, weighted graph, graph theory, greedy algorithm, network routing

1. Introduction

In any network of interconnected points — road intersections, internet routers, or communication nodes — multiple paths typically exist between a given source and destination, each carrying a different associated cost such as distance, latency, or monetary charge. The routing problem asks: of all possible paths between two nodes, which has the minimum total cost? This project, undertaken as a Semester 5 mini project under Design and Analysis of Algorithms, implements a network route optimizer using Dijkstra's algorithm, a classical greedy algorithm that solves the single-source shortest path problem for graphs with non-negative edge weights.

2. Theory: The Shortest Path Problem

A network is modeled as a weighted graph $G = (V, E)$, where V is the set of nodes and E is the set of edges, each edge carrying a non-negative weight representing its cost. The shortest path problem seeks the path between a source node and a target node that minimizes the sum of edge weights along the path. Dijkstra's algorithm solves this by maintaining a running estimate of the shortest known distance to every node, initialized to zero for the source and infinity for all others, and greedily finalizing the closest unvisited node at each iteration until every reachable node has been processed.

3. Example Network

The worked example used throughout this paper is a four-node network with nodes A, B, C, and D, connected by four weighted edges as shown in Figure 1: A–B with cost 4, A–C with cost 2, B–D with cost 5, and C–D with cost 1. The goal is to find the minimum-cost path from source node A to destination node D.

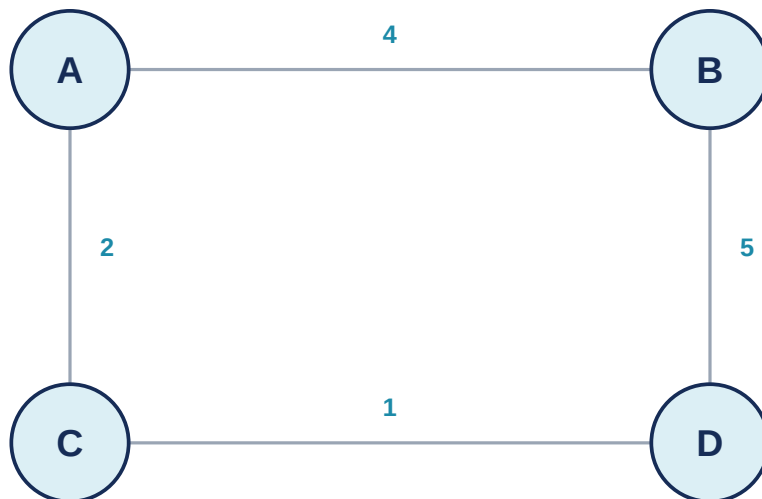


Figure 1. Example weighted network with four nodes and four edges.

4. Algorithm Steps

1. Assign the source node a distance of 0 and every other node a distance of infinity.
2. Select the unvisited node with the currently smallest known distance.
3. Update the distance of every neighboring node if a shorter path through the current node is found.
4. Mark the current node as visited and repeat until all reachable nodes have been visited.

5. Worked Example: Trace of Execution

Table 1 traces the algorithm's distance array through each iteration on the example network, starting from source node A.

Stage	A	B	C	D
Initial	0	∞	∞	∞
After visiting A	0	4	2	∞
After visiting C	0	4	2	3
After visiting D (final)	0	4	2	3

Table 1. Distance array evolution during Dijkstra's algorithm execution from source A.

The trace shows that after processing node C (distance 2 from A), the algorithm discovers that reaching D via C costs $2 + 1 = 3$, which is cheaper than reaching D via B at $4 + 5 = 9$. The algorithm therefore finalizes D at distance 3, and the resulting shortest path is $A \rightarrow C \rightarrow D$ with total cost 3, as highlighted in Figure 2.

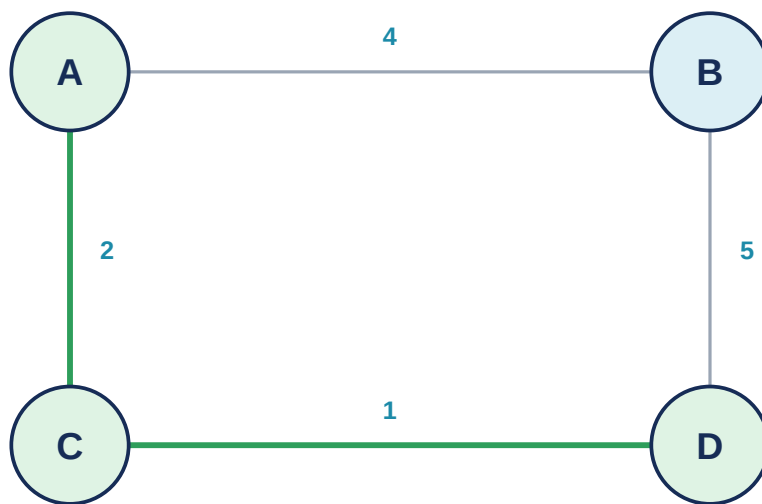


Figure 2. Shortest path $A \rightarrow C \rightarrow D$ (cost 3) highlighted in green against the full network.

6. C++ Implementation

The implementation represents the graph as a 4x4 adjacency matrix, where a nonzero entry at position $[i][j]$ denotes an edge of that weight between nodes i and j , and a zero entry denotes no direct edge. The algorithm maintains a distance array initialized to a large sentinel value (999, standing in for infinity) for all nodes except the source, and a boolean visited array. Each of the four outer iterations selects the unvisited node with minimum distance, marks it visited, and relaxes the distance of each of its neighbors.

```
int graph[4][4] = {
    {0,4,2,0},
    {4,0,0,5},
    {2,0,0,1},
    {0,5,1,0}
};

int dist[4] = {0,999,999,999};
bool visited[4] = {false};

for (int i = 0; i < 4; i++) {
    int min = 999, u;
    for (int j = 0; j < 4; j++)
        if (!visited[j] && dist[j] < min) {
            min = dist[j]; u = j;
        }

    visited[u] = true;

    for (int v = 0; v < 4; v++)
        if (graph[u][v] &&
            dist[u]+graph[u][v] < dist[v])
            dist[v] = dist[u]+graph[u][v];
}

cout << "Shortest Distance A->D = " << dist[3];
```

Listing 1. Array-based Dijkstra implementation. Sample output: Shortest Distance A->D = 3

7. Complexity Analysis

The array-based implementation shown above performs, for each of V nodes, a linear $O(V)$ scan to find the minimum-distance unvisited node, giving an overall complexity of $O(V^2)$. This is acceptable for small dense graphs but scales poorly for large sparse networks. Replacing the linear scan with a min-priority queue (typically a binary heap) reduces the complexity to $O((V+E) \log V)$, which is significantly faster for sparse graphs where the number of edges E is much smaller than V^2 . Table 2 summarizes both variants.

Implementation	Complexity	Notes
Basic Array	$O(V^2)$	Simple to implement; efficient only for small or dense graphs.
Priority Queue (Binary Heap)	$O((V+E) \log V)$	Preferred for large sparse graphs such as real-world road or router networks.

Table 2. Complexity comparison of Dijkstra implementations.

8. Applications

Dijkstra's algorithm underlies route computation in Google Maps and similar navigation tools, shortest-path packet forwarding in internet routers using link-state routing protocols such as OSPF, flight path optimization in airline networks, delivery route optimization for logistics companies, and adaptive traffic management in smart city infrastructure.

9. Limitations

Dijkstra's algorithm requires all edge weights to be non-negative; a negative edge weight can cause the algorithm to finalize a node's distance prematurely, before a cheaper negative-weight path through a later node is discovered, producing an incorrect result. Graphs with negative edges require alternative algorithms such as Bellman-Ford instead.

10. Result

A network route optimizer was successfully developed using Dijkstra's algorithm. The system correctly computed the shortest path between nodes on the example network, identifying $A \rightarrow C \rightarrow D$ with total cost 3 as the optimal route from A to D, and the accompanying interactive HTML tool reproduces this computation step by step against the same graph.

References

- Dijkstra, E. W. "A Note on Two Problems in Connexion with Graphs." *Numerische Mathematik*, 1959.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*, 3rd Edition. MIT Press, 2009.
- Sedgewick, R., and Wayne, K. *Algorithms*, 4th Edition. Addison-Wesley, 2011.
- GGSIPU B.Tech Computer Science Engineering Curriculum, Semester 5 — Design and Analysis of Algorithms course structure.

— Mayank Swaraj